

A Personalize é uma empresa de comunicação visual, letreiros e plotagens, de Belo Horizonte/ MG. Fui sócio da empresa por quase 9 anos e desenvolvi meus últimos trabalhos com marketing digital, Google, Instagram, e ciência de dados. No período em que trabalhei, houve um aumento em torno de 5 vezes no faturamento da Personalize.

O projeto dessa apresentação teve como foco a análise dos dados do setor comercial, principalmente em relação aos orçamentos gerados e vendas em decorrência dos anúncios no google ads, única forma de divulgação da empresa.

O maior obstáculo que enfrentei quando comecei a desenvolver o projeto foi como extrair os dados de valores gerados em orçamentos, vendas fechadas, tipo de serviço orçado pelo cliente, entre outros. No início eu conferia manualmente em cada conversa, mas como eram muitos contatos, acabava gastando 2 dias de serviço ou mais por mês só para gerar essa planilha, o que inviabilizou o processo por um tempo. Foi então que juntei o que havia estudado em ciência de dados, programação em python, NLP (Processamento de Linguagem Natural), e com ajuda do ChatGPT, consegui criar um código em python para extrair das conversas o que eu precisava para fazer as análises com os dados. Assim, diminuí o trabalho para 2 horas mensais, pois ainda tinha que exportar as conversas do whatsapp, principal meio de comunicação da empresa com os clientes. Mesmo assim, foi um ganho de tempo que permitiu elaborar novamente os painéis, gráficos, e extrair os insights para tomadas de decisão mais embasadas. Como por exemplo, a alocação do valor gasto com o google ads.

Abaixo segue o código em python que refinei para gerar a planilha dos dados:

```
import os
import re
import pandas as pd
import zipfile
from datetime import datetime, timedelta

# Caminho da pasta onde estão os arquivos .zip
PASTA_CONVERSAS = r"C:\Users\USER\OneDrive\Documentos\Trabalho..."

# Palavras-chave para identificar serviços e fechamento
palavras_servico = [
    "acrílico", "logo", "letreiro", "placa", "acrílico",
    "espelhado", "plotagem", "adesivo", "papel de parede", "PVC", "pvc", "luminoso"
]

palavras_fechado = [
    "vamos fazer", "aprovado", "pago", "pix feito",
```

```
"obrigada pelo pagamento", "obrigado pelo pagamento"  
]
```

```
# Função para normalizar serviços
```

```
def normalizar_servico(servico):  
    if servico.lower() == "adesivo":  
        return "plotagem"  
    if servico.lower() == "logo":  
        return "letrero"  
    if servico.lower() == "acrílico":  
        return "letrero"  
    if servico.lower() == "acrilico":  
        return "letrero"  
    if servico.lower() == "espelhado":  
        return "letrero"  
    if servico.lower() == "PVC":  
        return "letrero"  
    if servico.lower() == "pvc":  
        return "letrero"  
    if servico.lower() == "papel de parede":  
        return "plotagem"  
    return servico
```

```
# Função para extrair dados da conversa
```

```
def extrair_dados_conversa(texto):  
    data_contato = None  
    valor_orcado = None  
    servicos_encontrados = []  
    status = "Não"  
    telefone = None  
  
    # Extrair a primeira data encontrada  
    datas = re.findall(r"\d{1,2}/\d{1,2}/\d{2,4}", texto)  
    if datas:  
        for formato in ("%d/%m/%Y", "%d/%m/%y"):  
            try:  
                data_contato = datetime.strptime(datas[0], formato).date()  
                break  
            except:  
                continue  
  
    # Extrair valores orçados (maior valor)  
    valores = re.findall(r"R\$ ?[\d\.,]+", texto)  
    if valores:  
        valores_float = []
```

```

for v in valores:
    try:
        valores_float.append(float(v.replace("R$", "").replace(".", "").replace(",", ".")))
    except:
        pass
if valores_float:
    valor_orcado = max(valores_float)

# Extrair serviços mencionados
for palavra in palavras_servico:
    if palavra.lower() in texto.lower():
        servicos_encontrados.append(normalizar_servico(palavra))

# Verificar palavras que indicam fechamento
for p in palavras_fechado:
    if p.lower() in texto.lower():
        status = "Sim"
        break

return {
    "data_contato": data_contato,
    "valor_orcado": valor_orcado,
    "servicos": ", ".join(set(servicos_encontrados)) if servicos_encontrados else None,
    "status": status,
    "telefone": telefone
}

# Lista para armazenar os dados
dados = []

# Ler arquivos .zip da pasta
for arquivo in os.listdir(PASTA_CONVERSAS):
    if arquivo.lower().endswith(".zip"):
        caminho_zip = os.path.join(PASTA_CONVERSAS, arquivo)
        with zipfile.ZipFile(caminho_zip, 'r') as zip_ref:
            for nome in zip_ref.namelist():
                if nome.lower().endswith(".txt"):
                    with zip_ref.open(nome) as f:
                        texto = f.read().decode("utf-8", errors="ignore")
                        dados_extraidos = extrair_dados_conversa(texto)

                        # Extrair número de telefone ou nome do arquivo como fallback
                        telefone_encontrado = re.findall(r"\d{2,3}\s?\d{4,5}-?\d{4}",
os.path.basename(nome))

```

```

        telefone = telefone_encontrado[0] if telefone_encontrado else
os.path.basename(nome).replace("Conversa do WhatsApp com ", "").strip()
        dados_extraidos["telefone"] = telefone

    dados.append(dados_extraidos)

# Criar DataFrame
df = pd.DataFrame(dados)

# Converter data para datetime
df["data_contato"] = pd.to_datetime(df["data_contato"], errors="coerce")

# Definir limite para Cliente Antigo (mais de 2 meses)
hoje = datetime.today()
limite_antigo = pd.Timestamp(hoje - timedelta(days=60))

# Criar coluna tipo_cliente
df["tipo_cliente"] = df["data_contato"].apply(
    lambda d: "Cliente Antigo" if pd.notnull(d) and pd.Timestamp(d) < limite_antigo
    else "Cliente Novo"
)

# Filtrar clientes que tiveram orçamento no último mês
df = df[
    (df["valor_orcado"].notna()) &
    (df["data_contato"].dt.month == ultimo_mes) &
    (df["data_contato"].dt.year == ano_ultimo_mes)
]

# Remover duplicados (mantendo o primeiro registro por telefone)
df = df.drop_duplicates(subset=["telefone"], keep="first")

# Reorganizar colunas
df = df[["telefone", "data_contato", "servicos", "valor_orcado", "status", "tipo_cliente"]]

# Salvar em Excel
df.to_excel("relatorio_vendas_personalize.xlsx", index=False)

print("✅ Extração concluída! Arquivo salvo como 'relatorio_vendas_personalize.xlsx'")
print(f"Total de contatos únicos: {len(df)}")

```

Rodando esse código eu consegui obter a planilha que precisava. Coloquei os dados obtidos no Google Docs, para fazer o link com o Google Locker Studio, ferramenta de criação de dashboards. Assim, elaborei o seguinte relatório para apresentar para a diretoria:



O principal insight gerado, que foi passado para a diretoria, foi o grande valor de orçamentos enviados de plotagem, que converteram menos em vendas do que os letreiros. Assim, o comercial estava gastando muito tempo para elaborar orçamentos de plotagem, mais trabalhosos, e o que estava gerando mais receita eram os letreiros. Dessa forma, ficou mais claro em qual área a Personalize deveria focar e investir.

Esse foi um exemplo de informações levantadas que foram úteis para as tomadas de decisão. Foram observados outros pontos de atenção também, e com o acompanhamento mensal seria possível gerar relatórios cada vez mais embasados e aprofundados.